

UNIVERSITÀ
DEGLI STUDI
FIRENZE

Elaborato Calcolo Numerico 2024-2025

Autore: Nome Cognome
Matricola: XXXXXXX
E-mail: nome.cognome@edu.unifi.it

Giugno 2025

Indice

-Esercizio 1

Dimostrare che la formula alle differenze finite $\dfrac{25f(x) - 48f(x-h) + 36f(x-2h) - 16f(x-3h) + 3f(x-4h)}{12h} = f'(x) + O(h^4)$

Soluzione

Sviluppiamo in serie di Taylor centrate in x le funzioni f valutate nei punti $x - kh$ per $k = 1, 2, 3, 4$:

$$f(x-h) = f(x) - hf'(x) + \frac{h^2}{2}f''(x) - \frac{h^3}{6}f^{(3)}(x) + \frac{h^4}{24}f^{(4)}(x) + O(h^5)$$

$$f(x-2h) = f(x) - 2hf'(x) + 2h^2f''(x) - \frac{4h^3}{3}f^{(3)}(x) + \frac{2h^4}{3}f^{(4)}(x) + O(h^5)$$

$$f(x-3h) = f(x) - 3hf'(x) + \frac{9h^2}{2}f''(x) - \frac{9h^3}{2}f^{(3)}(x) + \frac{27h^4}{8}f^{(4)}(x) + O(h^5)$$

$$f(x-4h) = f(x) - 4hf'(x) + 8h^2f''(x) - \frac{32h^3}{3}f^{(3)}(x) + \frac{32h^4}{3}f^{(4)}(x) + O(h^5)$$

Sostituiamo nella formula e raccogliamo i coefficienti di ogni termine. Indicando con c_k il coefficiente moltiplicativo di $f(x - kh)$: $c_0 = 25$, $c_1 = -48$, $c_2 = 36$, $c_3 = -16$, $c_4 = 3$.

- **Coefficiente di $f(x)$**: $25 - 48 + 36 - 16 + 3 = 0$ ✓
- **Coefficiente di $hf'(x)$**: $0 + 48 - 72 + 48 - 12 = 12$ ✓
- **Coefficiente di $h^2f''(x)$**: $0 - 24 + 72 - 72 + 24 = 0$ ✓
- **Coefficiente di $h^3f^{(3)}(x)$**: $0 + 8 - 48 + 72 - 32 = 0$ ✓
- **Coefficiente di $h^4f^{(4)}(x)$**: $0 - 2 + 24 - 54 + 32 = 0$ ✓

Pertanto il numeratore si riduce a $12hf'(x) + O(h^5)$, e dividendo per $12h$:

$$\frac{25f(x) - 48f(x-h) + 36f(x-2h) - 16f(x-3h) + 3f(x-4h)}{12h} = f'(x) + O(h^4) \quad \square$$

-Esercizio 2

Data $f(x) = 1 + x^2 + \frac{\log|3(1-x)+1|}{80}$ *con asintoto verticale verso* $-\infty$ *in* $x = \frac{4}{3}$*, creare un grafico su MATLAB usando* `x = linspace(1, 5/3, 100001)` .

Codice MATLAB

```
% scriptExe2.m — Grafico funzione con asintoto verticale
x = linspace(1, 5/3, 100001);
f = @(x) 1 + x.^2 + log(abs(3*(1 - x) + 1)) / 80;

figure;
plot(x, f(x), 'b-', 'LineWidth', 1.5);
xlabel('x');
```

```
ylabel('f(x)');
title('Grafico di f(x) con asintoto in x = 4/3');
grid on;
ylim([1.8, 4]);
```

Osservazioni

Il grafico presenta un picco verso il basso in corrispondenza dell'asintoto in $x = \frac{4}{3} \approx 1.333$. Poiché si impiega un numero finito di punti (100001), l'asintoto non viene rappresentato fedelmente: si osserva solo un minimo locale nel punto $(1.33333, 2.3359)$. Il minimo globale — che analiticamente non esiste per la presenza dell'asintoto verso $-\infty$ — risulta invece nel punto $(1, 2)$.

⚠️ Eseguire `scriptExe2.m` per visualizzare il grafico interattivo.

-Esercizio 3

Spiegare in modo esaustivo il fenomeno della cancellazione numerica e fornire un esempio illustrativo.

La **cancellazione numerica** si verifica quando si sottrae una quantità a un'altra di valore molto simile, causando la perdita di cifre significative nel risultato.

Analisi tramite numero di condizionamento

Per la somma di due operandi x_1 e x_2, il numero di condizionamento è:

$$k = \frac{|x_1| + |x_2|}{|x_1 + x_2|}$$

Quando $x_1 \approx -x_2$, il denominatore si avvicina a zero e k cresce senza limite, amplificando enormemente l'errore relativo sul risultato.

Esempio: formula quadratica instabile

Consideriamo le radici di $x^2 - 56x + 1 = 0$.

La formula standard dà $x_1 = 28 + \sqrt{783} \approx 55.982$ e $x_2 = 28 - \sqrt{783} \approx 0.01786$.

Il calcolo di x_2 con la formula diretta soffre di cancellazione. La formula stabile usa la relazione $x_1 \cdot x_2 = 1$:

```
a = 1; b = -56; c = 1;
x1 = (-b + sqrt(b^2 - 4*a*c)) / (2*a);   % stabile
x2_instabile = (-b - sqrt(b^2 - 4*a*c)) / (2*a);  % cancellazione!
x2_stabile   = c / (a * x1);              % formula alternativa stabile

fprintf('x2 instabile: %.15f\n', x2_instabile);
fprintf('x2 stabile:   %.15f\n', x2_stabile);
```

-Esercizio 4

Scrivere una function MATLAB che implementi in modo efficiente il metodo di bisezione.

Implementazione

```
function [xstar, itnum] = metodoBisezione(f, a, b, tol)
% [xstar, itnum] = metodoBisezione(f, a, b, tol)
% Calcola una radice di f in [a,b] con tolleranza tol.
%
% Input:
%   f      - function handle
%   a, b   - estremi dell'intervallo (f(a)*f(b) < 0)
%   tol    - tolleranza sull'errore  [default: 1e-3]
% Output:
%   xstar  - stima della radice
%   itnum  - numero di iterazioni eseguite

if nargin == 3, tol = 1e-3; end
if nargin < 3 || nargin > 4, error('Numero di argomenti errato'); end
if a >= b || tol <= 0,       error('Argomenti errati'); end

itnum = 0;
fa = feval(f, a);  fb = feval(f, b);

if fa == 0, xstar = a; return; end
if fb == 0, xstar = b; return; end
if fa * fb > 0, error('Intervallo errato: f(a)*f(b) > 0'); end

itmax = ceil(log2(b - a) - log2(tol));
for i = 1:itmax
   xstar = (a + b) / 2;
   itnum = itnum + 1;
   fx    = feval(f, xstar);
   if abs((fx * (b - a)) / (fb - fa)) <= tol, return; end
   if fa * fx < 0
      b = xstar; fb = fx;
   else
      a = xstar; fa = fx;
   end
end
end
```

Casi di test

1. **Matrice singolare** (intervallo con segni concordi) ⟶ errore corretto.
2. **Radice esatta in** `a` ⟶ restituzione immediata senza iterazioni.
3. **Tolleranza omessa** ⟶ valore di default `tol = 1e-3` applicato.

-Esercizio 5

Scrivere function MATLAB distinte che implementino il metodo di Newton e il metodo delle secanti per la ricerca degli zeri di $f(x)$.

Metodo di Newton

```
function [xstar, itnum] = metodoNewton(f, df, x0, tol, itmax)
% [xstar, itnum] = metodoNewton(f, df, x0, tol, itmax)
if nargin == 3, tol = 1e-3; itmax = 1e3; end
if nargin == 4, itmax = 1e3; end
if tol <= 0 || itmax <= 0, error('Argomenti errati'); end

flag = 0;  itnum = 0;
for i = 1:itmax
   f0  = feval(f,  x0);
   df0 = feval(df, x0);
   if f0  == 0, xstar = x0; return; end
   if df0 == 0
      warning('Derivata prima nulla — metodo interrotto'); return;
   end
   xstar  = x0 - f0 / df0;
   itnum  = itnum + 1;
   if abs(xstar - x0) <= tol * (1 + abs(xstar))
      flag = 1;  break;
   end
   x0 = xstar;
end
if ~flag, warning('Tolleranza non raggiunta'); end
end
```

Metodo delle Secanti

```
function [xstar, itnum] = metodoSecanti(f, x0, x1, tol, itmax)
% [xstar, itnum] = metodoSecanti(f, x0, x1, tol, itmax)
if nargin == 3, tol = 1e-3; itmax = 1e3; end
if nargin == 4, itmax = 1e3; end
if tol <= 0 || itmax <= 0 || x0 == x1, error('Argomenti errati'); end

flag = 0;  itnum = 0;
fx0 = feval(f, x0);  fx1 = feval(f, x1);
if fx0 == 0, xstar = x0; return; end
if fx1 == 0, xstar = x1; return; end

for i = 1:itmax
   if fx0 == fx1, error('Il metodo non converge'); end
   xstar  = (fx1*x0 - fx0*x1) / (fx1 - fx0);
   itnum  = itnum + 1;
   if abs(xstar - x1) <= tol * (1 + abs(xstar))
      flag = 1;  break;
   end
   x0 = x1;   x1 = xstar;
   fx0 = fx1; fx1 = feval(f, xstar);
```

```
end
if ~flag, warning('Tolleranza non raggiunta'); end
end
```

-Esercizio 6

Usare le function dei precedenti esercizi per approssimare la radice di $f(x) = e^x - \cos(x)$, partendo da $x_0 = 1$ (e $x_0 = 0.9$ per le secanti) con intervallo $[-0.1, 1]$ per la bisezione. Tabulare risultati e costo computazionale.

La radice di $f(x) = e^x - \cos(x)$ è $x^* = 0$.

Tab. 1 — Approssimazioni della radice per tolleranza

	10^{-3}	10^{-6}	10^{-9}	10^{-12}
Bisezione	9.77×10^{-4}	9.54×10^{-7}	9.31×10^{-10}	9.09×10^{-13}
Newton	2.84×10^{-9}	3.57×10^{-17}	3.57×10^{-17}	3.57×10^{-17}
Secanti	1.15×10^{-6}	2.09×10^{-16}	2.09×10^{-16}	-1.26×10^{-17}

Tab. 2 — Numero di valutazioni di funzione per tolleranza

	10^{-3}	10^{-6}	10^{-9}	10^{-12}
Bisezione	9	19	29	39
Newton	5	6	6	6
Secanti	6	8	8	9

Commento

Newton converge quadraticamente e raggiunge la precisione macchina già per tol $= 10^{-6}$. Il metodo delle secanti, con ordine di convergenza superlineare (≈ 1.618), ha un costo per iterazione minore di Newton (non richiede la derivata): risulta il più efficiente per valutazioni di funzione totali. La bisezione è lineare: il numero di iterazioni cresce di circa 10 per ogni ordine di grandezza in tolleranza.

⚠️ Per il dettaglio numerico vedere `esercizio6.txt` o eseguire `scriptExe6.m`.

-Esercizio 7

Scrivere una function MATLAB `mialu` che, data in ingresso una matrice A e un vettore b, calcoli la soluzione di $A\boldsymbol{x} = \boldsymbol{b}$ tramite fattorizzazione LU con pivoting parziale.

Implementazione

```
function x = mialu(A, b)
% x = mialu(A, b)
% Risolve Ax = b con fattorizzazione LU con pivoting parziale.
% A: matrice quadrata nxn non singolare; b: vettore colonna nx1.

[m, n] = size(A);
if m ~= n || ~iscolumn(b) || m ~= length(b)
   error('Argomenti non compatibili');
end

tol = n * eps * norm(A, inf);
p   = 1:n;

% ── Fattorizzazione LU con pivoting parziale ──────────────────
for j = 1:n
   [mj, kj] = max(abs(A(j:n, j)));
   if mj <= tol, error('Matrice A singolare'); end
   kj = kj + j - 1;
   if kj > j
      A([j kj], :) = A([kj j], :);
      p([j kj])    = p([kj j]);
   end
   A(j+1:n, j)     = A(j+1:n, j) / A(j, j);
   A(j+1:n, j+1:n) = A(j+1:n, j+1:n) - A(j+1:n,j) * A(j, j+1:n);
end

% ── Risoluzione Ly = Pb ──────────────────────────────────────────
x = b(p);
for j = 1:n-1
   x(j+1:n) = x(j+1:n) - x(j) * A(j+1:n, j);
end

% ── Risoluzione Ux = y ──────────────────────────────────────────
for j = n:-1:1
   x(j)     = x(j) / A(j, j);
   x(1:j-1) = x(1:j-1) - x(j) * A(1:j-1, j);
end
end
```

Test significativi

1. $\boldsymbol{A}$ **singolare** (righe duplicate) $\longrightarrow$ errore corretto.
2. $\boldsymbol{A}$ **non quadrata** $\longrightarrow$ errore corretto.
3. $\boldsymbol{b}$ **non colonna** $\longrightarrow$ errore corretto.
4. **Caso regolare** $\longrightarrow$ soluzione con residuo $\|A\boldsymbol{x} - \boldsymbol{b}\| \approx 10^{-15}$.

⚠️ Eseguire `scriptExe7.m` o consultare `esercizio7.txt` per tutti i test.

-Esercizio 8

Costruire una function `lagrange.m` *avente la stessa sintassi di* `spline`*, che implementi in modo vettoriale la forma di Lagrange del polinomio interpolante.*

Formula di Lagrange

Il polinomio interpolante di grado n passante per i punti $(x_0, y_0), \ldots, (x_n, y_n)$ è:

$$p(x) = \sum_{j=0}^{n} y_j \ell_{j(x)}, \quad \ell_{j(x)} = \prod_{k=0, k\neq j}^{n} \frac{x - x_k}{x_j - x_k}$$

Implementazione

```
function s = lagrange(x, y, xq)
% s = lagrange(x, y, xq)
% Valuta il polinomio interpolante di Lagrange nei punti xq.
if nargin ~= 3, error('Numero di argomenti errato'); end
lx = length(x);
if lx ~= length(y), error('Argomenti errati'); end

s = zeros(length(xq), 1);
for i = 1:length(xq)
   for j = 1:lx
      temp = 1;
      for k = 1:lx
         if k ~= j
            if x(k) == x(j), error('Ascisse non distinte'); end
            temp = temp * ((xq(i) - x(k)) / (x(j) - x(k)));
         end
      end
      s(i) = s(i) + temp * y(j);
   end
end
end
```

La complessità è $O(n^2)$ per ogni punto di valutazione. Si noti che la forma di Newton (con differenze divise) risulta più stabile numericamente per gradi elevati.

-Esercizio 9

Scrivere una function `composita(fun, a, b, k, n)` *che implementi la formula composita di Newton–Cotes di grado k su $n+1$ ascisse equidistanti, con n multiplo pari di k.*

Pesi per gradi $k = 1, 2, 3$

Grado k	Nome	Pesi c_i	Errore
$k = 1$	Trapezi	$\frac{1}{2}$,; $\frac{1}{2}$	$O(h^2)$
$k = 2$	Simpson	$\frac{1}{6}$,; $\frac{4}{6}$,; $\frac{1}{6}$	$O(h^4)$

Grado k	Nome	Pesi c_i	Errore
$k=3$	Regola 3/8	$\frac{1}{8}$,; $\frac{3}{8}$,; $\frac{3}{8}$,; $\frac{1}{8}$	$O(h^4)$

Implementazione

```
function [If, err] = composita(fun, a, b, k, n)
% [If, err] = composita(fun, a, b, k, n)
% Formula composita di Newton-Cotes grado k, n+1 nodi equidistanti.
if nargin ~= 5, error('Numero di argomenti errato'); end
if a > b || mod(n, k) ~= 0, error('Argomenti errati'); end

% Calcolo pesi per la formula di grado k
kk = k+1:-1:1;
c  = zeros(1, k+1);
for i = 0:k
   v       = [0:i-1, i+1:k];
   den     = prod(i - v);
   ap      = poly(v);
   c(i+1)  = polyval([ap ./ kk, 0], k) / den;
end

xi = linspace(a, b, n+1);
yi = feval(fun, xi);  % valutazione vettoriale

% Somma composita (bordi contati una sola volta)
If = 0;
for i = 1:n+1
   ind = mod(i-1, k);
   if ind == 0 && i ~= 1 && i ~= n+1
      If = If + 2 * yi(i) * c(ind+1);
   else
      If = If + yi(i) * c(ind+1);
   end
end
If = If * ((b - a) / n);

% Stima errore con estrapolazione di Richardson
p = n / k;  If2 = 0;
for i = 1:p:n+1
   ind = mod((i-1)/p, k);
   If2 = If2 + yi(i) * c(ind+1);
end
If2 = If2 * ((b - a) / k);
if mod(k, 2) == 0
   err = abs((If - If2) / (p^(k+2) - 1));
else
   err = abs((If - If2) / (p^(k+1) - 1));
end
end
```

Risultati per $I(f) = \int_0^1 e^{3x}\,\mathrm{d}x = \frac{e^3-1}{3} \approx 6.3619$, $n = 12$

k	Integrale stimato	Errore stimato	Errore vero
$k = 1$	6.39495	2.90×10^{-2}	3.31×10^{-2}
$k = 2$	6.36198	1.08×10^{-4}	1.37×10^{-4}
$k = 3$	6.36215	2.50×10^{-4}	3.06×10^{-4}
$k = 6$	6.36185	7.77×10^{-8}	9.63×10^{-8}